

ITERATION 4 FINAL · GOOGLE DOC ALIGNED

9개 Design Pattern을 아주 구체적으로 설명하는 발표 Deck

Google Doc의 Form #1~#3, Section 4~16을 반영하고,
실제 코드와 맞지 않는 표현은 정정해 구조/메서드/시연 흐름을 다시 정리했다.

이번 v3 기준

페이지 수를 아끼지 않음

1. 누적 요구사항
2. 변경/확장 목록
3. DP별 레퍼런스 구조
4. 우리 클래스 매핑
5. 실제 코드 skeleton
6. 앱 시연 순서

State

Strategy

Observer

Composite

Singleton

Factory

Template

Adapter

Decorator

수정 원칙: 문서 그대로 + 코드 대조

Google Doc 내용은 최대한 가져오되, 실제 코드와 어긋나는 부분은 발표에서 틀리지 않게 정정한다.

1. Google Doc 반영

Form #1~#3, Section 4~12, Section 13~16 내용을 deck
장표로 확장

2. 코드 기준 검증

로컬 src의 실제 클래스/메서드를 확인해 DP 역할 매핑
수정

3. 발표 가능성

기능 설명보다 구조도, skeleton, demo trigger를
강화

오류 위험	문서 표현	deck에서 정정할 표현
State	구체 상태가 interface 직접 구현	현재 코드는 AbstractReservationState 기본 거부층 포함
Factory Method	PaymentProcessorFactory만 Creator처럼 보임	PaymentMethodProcessor와 ItineraryFactory가 더 교과서적 Creator
UI migration	Control/Domain 한 줄도 변경 없음	Boundary 중심 전환 + Control glue 추가, Domain 패턴 보존

Source Map: Google Doc → Deck

사용자가 준 링크의 실제 export 내용을 기준으로 장표를 배치했다.

Google Doc 위치	주요 내용	deck 반영 위치
Form #1	누적 기능 인벤토리	Feature Inventory 2장
Form #2	확장/변경 목록	Extension Inventory
Form #3	Refactoring / DP Inventory	R/DP Mapping
Section 4~6	Scope, migration, feature breakdown	Architecture + demo trigger
Section 7~9	UML/sequence/demo scenarios	Diagram/sequence/runbook
Section 10~12	QA, limits, progress	QA/limits/progress
Section 13~16	iter1~3 recap + Pattern Reference	DP별 레퍼런스/팀 구현 1:1

Form #1 Feature Inventory — Auth / Search

누적 기능을 기능군별로 쪼개서 실제 데모 지점과 연결한다.

Category	Sub-feature	Iter	DP / Demo link
Authentication	Member registration	1,4 UI	회원 가입 화면
Authentication	Login / Logout (Skypass member)	1	세션 유지
Authentication	Member profile + mileage balance	2	Adapter demo precondition
Authentication	Guest verification (PNR + name + email)	2	Guest lookup
Flight Search	Flight search by origin/destination/date/trip type	1	SearchController
Flight Search	Direct-flight selection	1	direct booking
Flight Search	Connecting-flight selection with MCT	3,4 UI	ItineraryFactory
Flight Search	Multi-city itinerary composition	3,4 UI	MultiCityItineraryFactory
Flight Search	City-code search expanding to airports	4	Composite

Form #1 Feature Inventory — Booking / Mileage / Ticket

예약 흐름에서 iter4 신규 DP가 실제 UI 행동으로 드러난다.

Category	Sub-feature	Iter	DP / Demo link
Booking	Passenger info entry	1	Passenger screen
Booking	Seat selection	2	Seat screen
Booking	Seat add-on upgrades: legroom / lounge	4	Decorator
Booking	15-minute seat hold management	3	Observer / scheduler
Booking	Multi payment-method routing	4	Factory Method
Booking	Auto-cancel on payment failure	3	Observer + State
Mileage	Mileage balance lookup via Skypass	3,4 UI	Adapter
Mileage	Partial/full mileage redemption	3,4 UI	Adapter + Factory
e-Ticket	Multi-format render: text / HTML / boarding pass	4	Template Method
Notifications	Linked bus-ticket offer on e-ticket issuance	3	Observer
Options	Global config: font, currency, theme	4	Singleton

Form #2 Extension Inventory

iteration 4에서 완료한 확장과 의도적 후속 과제를 구분한다.

Base feature	Extension	Status in Iteration 4
Itinerary composition	Direct / Connecting / MultiCity factories	Done · DP#6 ItineraryFactory
Options and Settings	Font, currency, theme global config	Done · DP#5 Singleton AppConfig
Payment methods	KakaoPay, ApplePay, bank, mileage	Done · DP#6 Payment processors
e-Ticket issuance	Multi-format rendering	Done · DP#7 TicketRenderer
External Skypass	Wrap mileage API behind domain interface	Done · DP#8 Adapter
Seat selection	Per-seat add-on options stacked at runtime	Done · DP#9 Decorator
Connecting search	Same-city different-airport grouping	Done · DP#4 Composite
UI toolkit	Swing replaced with JavaFX	Done · Refactoring#3
Exceptional refund	Admin review queue with approve/deny	Partial · backend auth stub
Mileage accrual / PDF	Accrual and localized PDF download	Post-course

Form #3 Refactoring / DP Inventory

각 iteration에서 refactoring이 DP로 발전한 흐름을 보여준다.

Iter	Type	Where applied / Summary	발표 포인트
1st	DP#1 State	Reservation lifecycle: State interface + concrete states	상태별 허용 전이
2nd	Refactoring#1	RefundHandler if/else extracted to RefundPolicy	Strategy 기반 마련
2nd	DP#2 Strategy	Full / Partial / No refund policies	paid 계산식 분리
3rd	Refactoring#2	Side-effect calls moved to publisher-listener	Observer 기반 마련
3rd	DP#3 Observer	EventPublisher + EventListener + DomainEvent	1-to-N 부수효과 분리
4th	Refactoring#3	Swing → JavaFX boundary migration	Boundary 중심 전환
4th	DP#4~#9	Composite, Singleton, Factory, Template, Adapter, Decorator	신규 6개 DP
Note	번호 정정	iter3 preview의 DP#4/5 계획 번호와 구현 번호 다름	Composite=DP#4, Singleton=DP#5

Section 4 Scope Overview

iter4는 UI 노출과 신규 DP 추가가 동시에 일어난 최종 통합 단계다.

한 문장 요약

Swing UI에서 닿지 않던 결제수단 선택, 좌석 부가옵션, e-ticket 렌더링, 마일리지 결제, 환승 검색, 전역 설정을 JavaFX 화면으로 끌어올렸다.

동시에 추가된 DP

Composite · Singleton · Factory Method · Template Method · Adapter · Decorator

기존 State · Strategy · Observer와 합쳐 총 9개

Area	Iteration 3	Iteration 4
Main patterns	Observer	Composite / Singleton / Factory / Template / Adapter / Decorator
UI toolkit	Swing imperative	JavaFX FXML + CSS + MVC
Patterns exercised in UI	State, Strategy, Observer	All 9 patterns

Section 4.3 Code Change Footprint — 정정된 표현

Google Doc 표현을 그대로 말하면 오해가 생길 수 있어 실제 코드 기준으로 표현을 다듬는다.

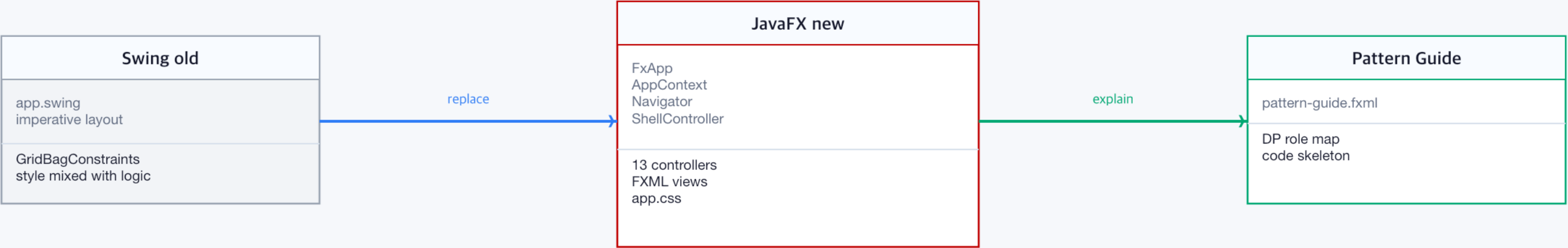
Layer	Google Doc 내용	발표용 정정
Boundary	Swing 19 files removed, app.fx package added	맞음: JavaFX FXML/CSS/controllers 추가
Control	confirmPaymentWith, initiateConnectingBooking added	맞음: UI 노출을 위한 thin glue 추가
Domain	Unchanged	주의: 패턴 클래스는 Domain/Control에 존재. 'UI migration이 패턴 구조를 깨지 않았다'로 말하기
Build	pom.xml, mvnw added	Maven + javafx-maven-plugin 기준으로 실행

권장 발표 문장

“UI 전환 자체는 Boundary 중심이었고, Control/Domain의 핵심 패턴 구조는 보존했습니다. 다만 iter4 신규 기능을 노출하기 위한 얇은 control glue와 패턴 클래스는 추가되었습니다.”

Section 5 UI Stack Migration

FXML, CSS, Controller로 화면 구조와 스타일, 로직을 분리했다.



구성	발표 포인트
FXML	화면 구조를 선언적으로 표현
CSS	대한항공 light-theme 스타일 분리
Controller	화면 이벤트가 Control/Domain DP 호출로 이어짐

Section 6 Feature Breakdown — Demo Trigger

각 신규 기능을 어떤 화면 행동으로 보여줄지 미리 고정한다.

#	Feature	Demo trigger	Pattern
1	Payment-method routing	Choose a method on payment screen	Factory Method
2	Mileage balance + payment	Member payment screen shows balance	Adapter
3	Seat add-on surcharge	Tick legroom / lounge on seat screen	Decorator
4	Multi-format e-Ticket	Toggle format on confirmation screen	Template Method
5	Global settings	Change font / currency in Settings	Singleton
6	City-code search	Search by SEL / TYO / NYC / LON	Composite
7	Connecting search	Tick include connections	Factory Method
8	Manual refund review	Request review then approve / deny	Strategy

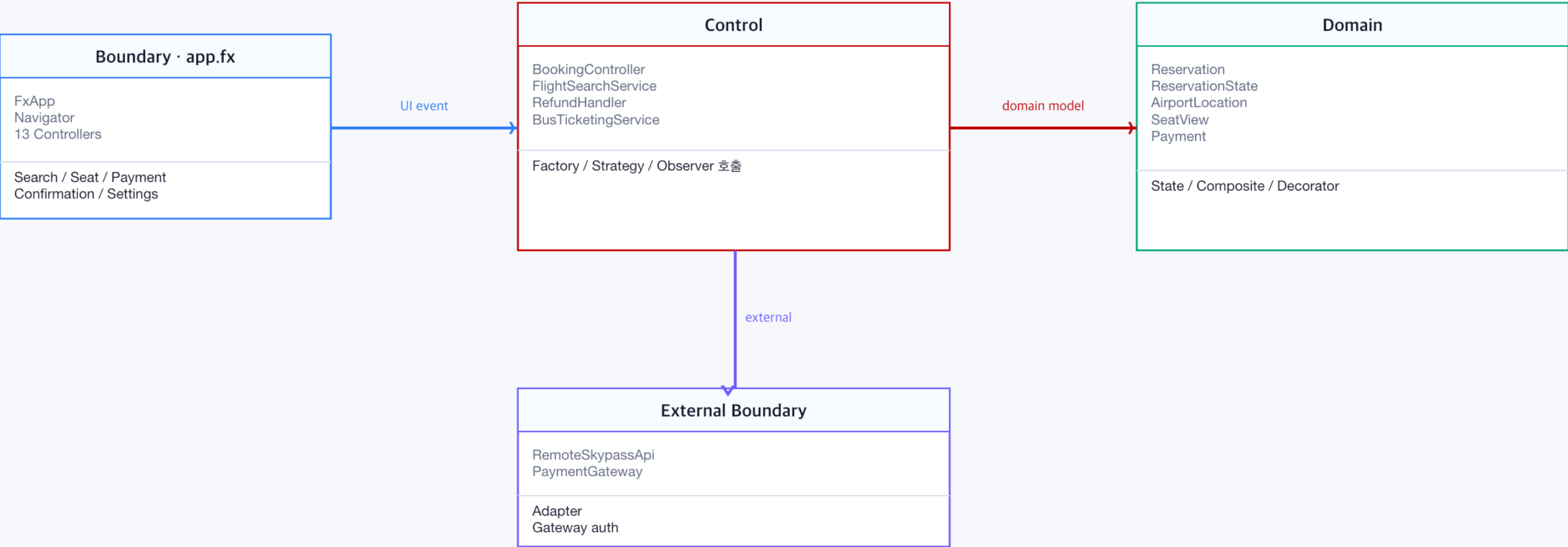
Section 7 Diagram Set — deck에서 보여줄 그림

Google Doc의 diagram placeholder를 실제 발표 장표 흐름으로 옮긴다.

Diagram group	Deck representation	설명 목적
Full cumulative class diagram	macro architecture map	Boundary/Control/Domain/External 경계
Per-pattern class diagrams	DP#1~#9 reference vs team	교과서 역할과 우리 클래스 1:1
Use case cumulative	red-marked iter4 use cases	새 UI 기능이 어디 붙었는지
State diagram	admin refund-review transition	iter4 review queue 흐름
Sequence diagrams	Adapter / Factory / Decorator	실제 화면 행동의 호출 순서

Cumulative Architecture Map

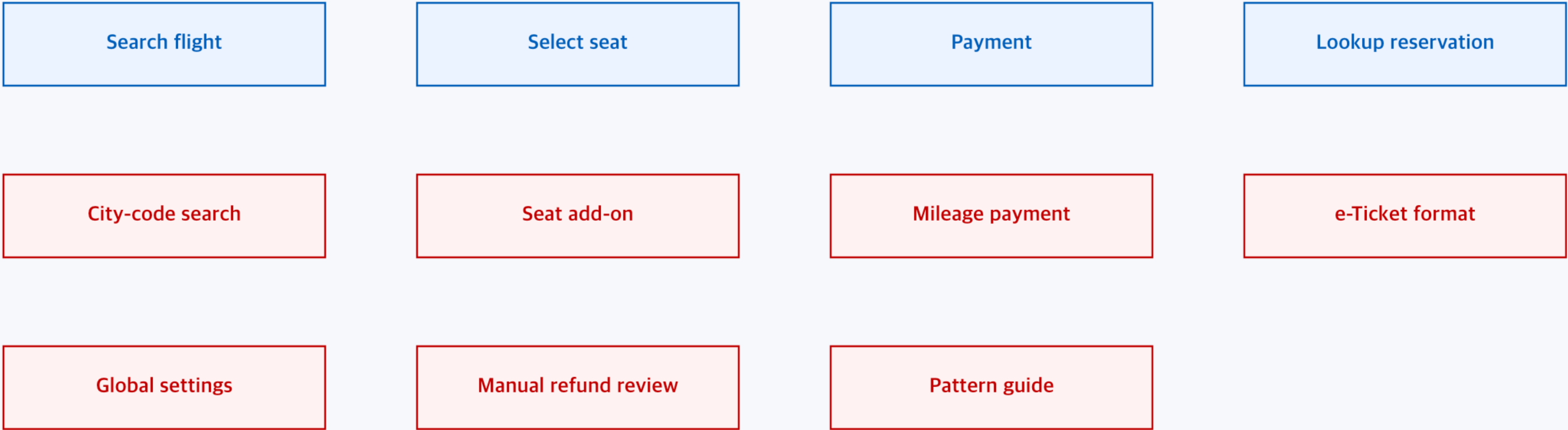
전체 클래스 다이어그램은 너무 크므로, 발표에서는 경계와 DP 위치를 먼저 보여준다.



DP 위치: State/Composite/Decorator=Domain, Strategy/Factory/Observer/Template=Control+Domain, Adapter=Boundary

Use Case Diagram — iter4 red additions

실제 다이어그램 대신 발표용 use case grouping으로 빨간 신규 기능을 표시한다.

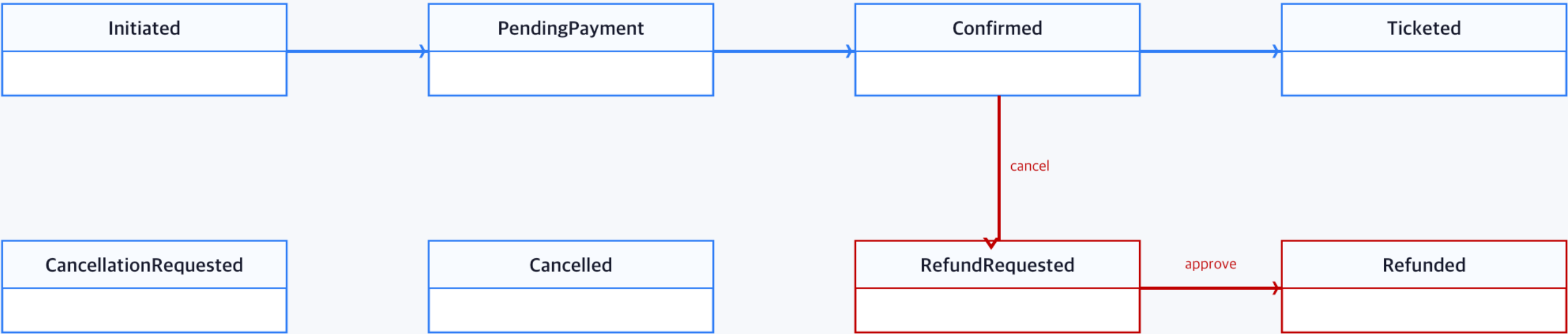


발표 포인트

파란색은 기존 예약 흐름, 빨간색은 iteration 4에서 UI로 새로 노출되거나 강화된 use case다.

State Diagram — admin refund-review transition

새 상태는 추가하지 않고 RefundRequested에서 승인/거절 분기만 시연한다.

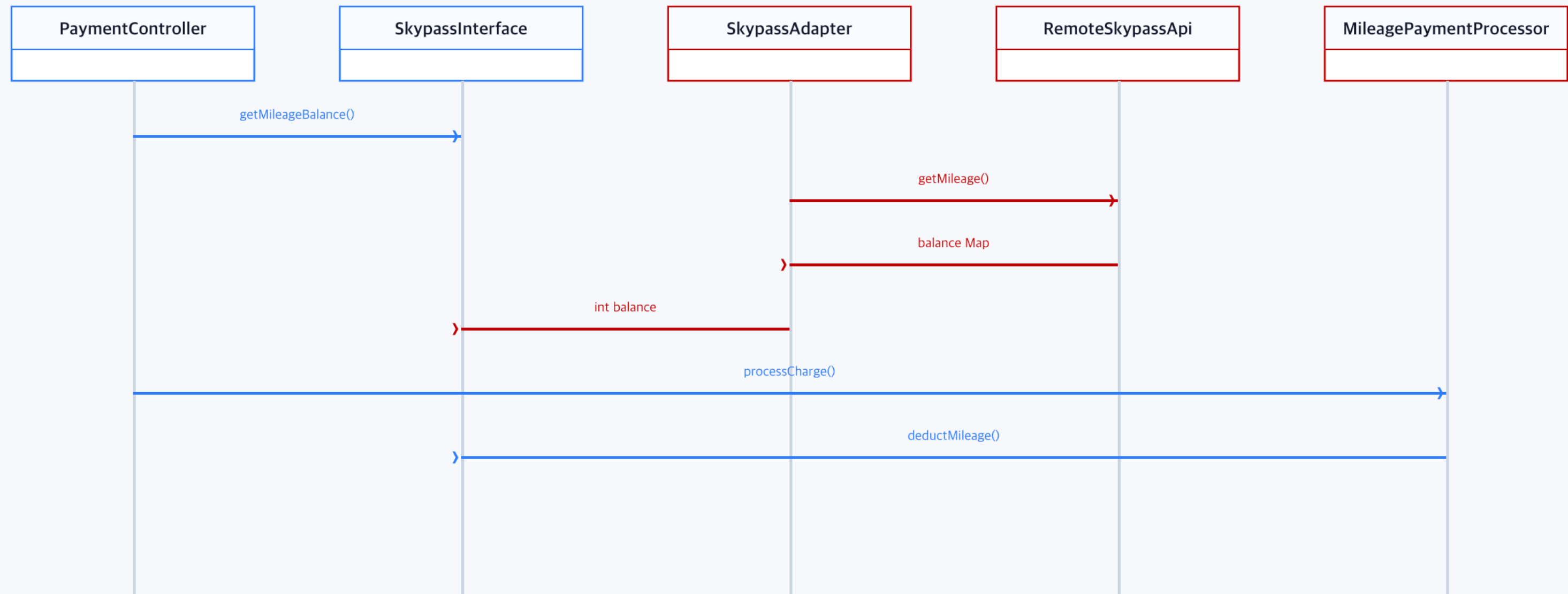


시연

예약 조회에서 취소/환불 요청 후 review queue로 이동한다. 승인 시 RefundRequested → Refunded, 거절 시 Cancelled 복귀를 설명한다.

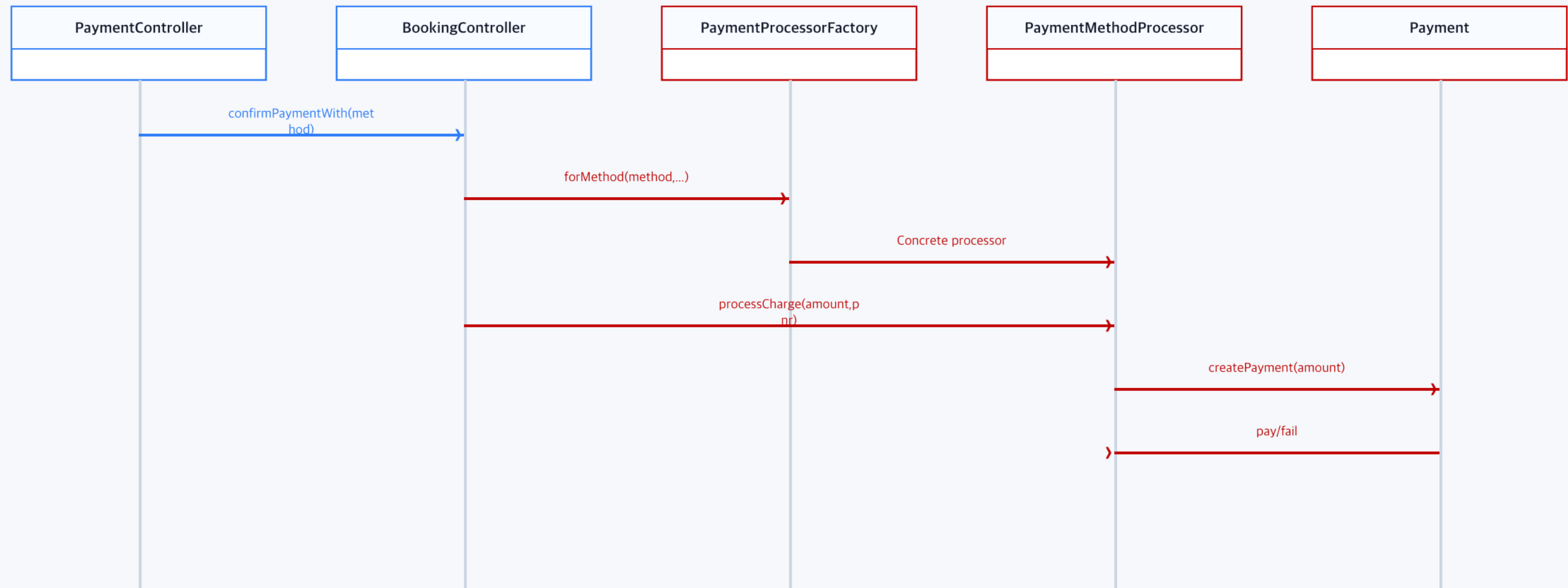
Sequence — Mileage Payment via Adapter

Payment screen은 SkypassInterface만 알고 외부 API raw Map은 Adapter가 숨긴다.



Sequence — Payment Method Routing via Factory Method

결제 수단 선택이 concrete processor 생성으로 이어진다.

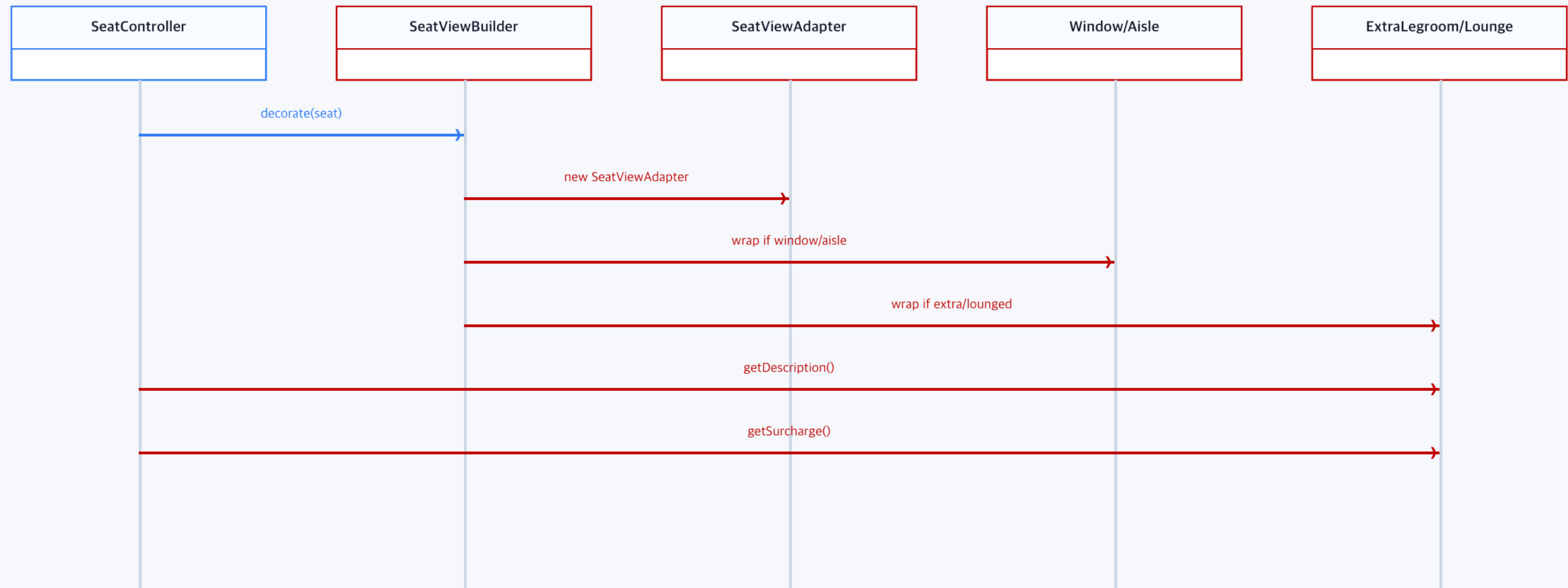


정정

교과서적 Factory Method 핵심은 PaymentMethodProcessor.createPayment()와 ItineraryFactory.createItinerary(); PaymentProcessorFactory는 선택 helper로 설명한다.

Sequence — Seat Add-on Stacking via Decorator

SeatViewBuilder가 seat 상태에 따라 wrapper를 누적한다.



Pattern Application Progress

각 패턴은 도입된 iteration 이후 계속 active로 유지된다.

Pattern	Category	Iter1	Iter2	Iter3	Iter4
State	Behavioral	* NEW	active	active	active
Strategy	Behavioral	-	* NEW	active	active
Observer	Behavioral	-	-	* NEW	active
Composite	Structural	-	-	-	* NEW
Singleton	Creational	-	-	-	* NEW
Factory Method	Creational	-	-	-	* NEW
Template Method	Behavioral	-	-	-	* NEW
Adapter	Structural	-	-	-	* NEW
Decorator	Structural	-	-	-	* NEW

Plan vs Delivered

Proposal #0 대비 4개 패턴을 초과 달성했다.

Pattern	Original plan	Delivered
State	Planned	iter1 done
Strategy	Planned	iter2 done
Observer	Planned	iter3 done
Singleton	Planned	iter4 done
Factory Method	Optional planned	iter4 done
Composite	Beyond plan	iter4 done
Template Method	Beyond plan	iter4 done
Adapter	Beyond plan	iter4 done
Decorator	Beyond plan	iter4 done
Total	5 planned	9 delivered (+4)

Iteration 1 Recap — State

happy path에서 시작한 예약 생애주기를 상태 객체로 캡슐화했다.

기능

login → search → passenger → payment → confirmation end-to-end happy path

DP#1 State

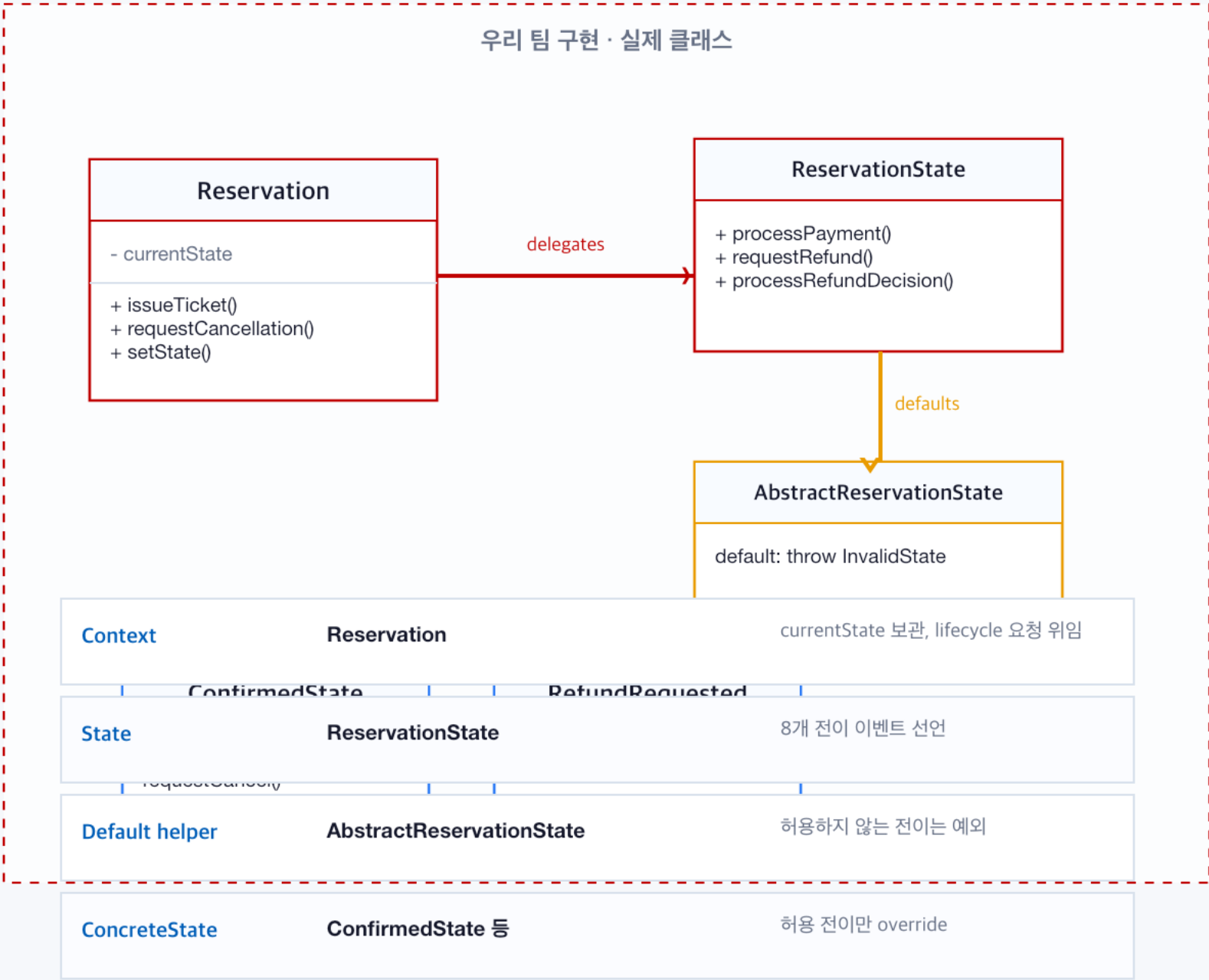
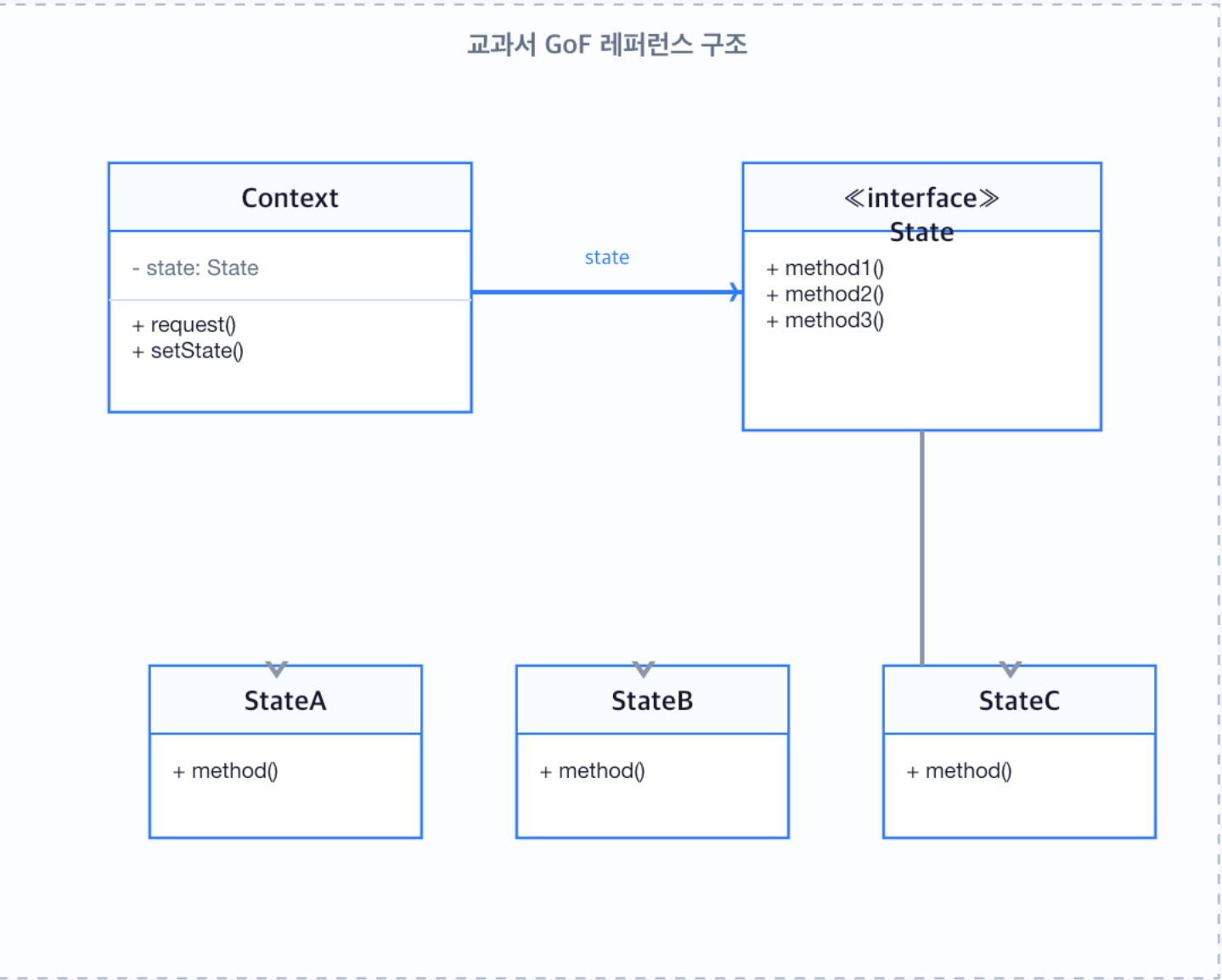
Reservation lifecycle을 enum+switch가 아니라
ReservationState interface와 concrete state로 분리

정정

Google Doc에는 'no abstract layer'처럼 보이는 문장이 있으나 현재 코드는 AbstractReservationState가 디폴트 거부 로직을 제공한다. 발표에서는 이 helper layer를 명시한다.

DP#1 State — Reference vs Team

Context가 현재 State에 요청을 위임하고, ConcreteState가 전이 가능 여부를 결정한다.



DP#1 State — Code Skeleton

State badge와 터미널 로그로 전이를 시연한다.

실제 코드 skeleton

```
interface ReservationState {
    void processPayment(Reservation ctx);
    void issueTicket(Reservation ctx);
    void requestCancellation(Reservation ctx);
    void processRefundDecision(Reservation ctx, boolean approved);
}

abstract class AbstractReservationState implements ReservationState {
    public void issueTicket(Reservation ctx) {
        throw new InvalidStateTransitionException(name(), "issueTicket");
    }
}

class ConfirmedState extends AbstractReservationState {
    public void issueTicket(Reservation ctx) {
        ctx.addTicket(Ticket.generate(...));
        ctx.setState(new TicketedState());
    }
}
```

논리 흐름



시연 멘트

결제 성공 후 Confirmed, 발권 후 Ticketed로 바뀌는 STATE 로그를 같이 보여준다.

Iteration 2 Recap — Strategy

환불 if/else를 RefundPolicy family로 분리했다.

기능

cancellation, refund, e-ticket issuance, reservation lookup, seat selection

DP#2 Strategy

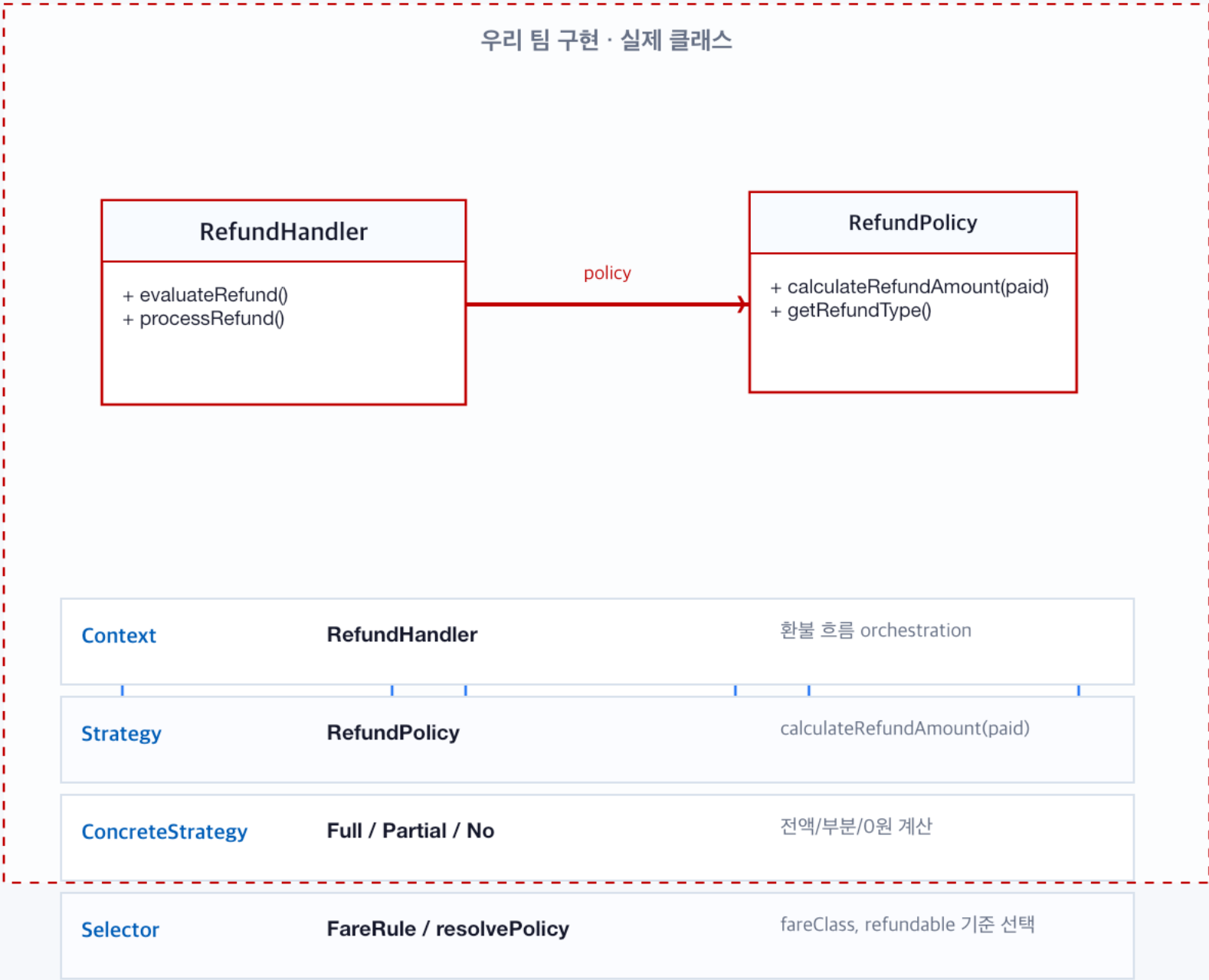
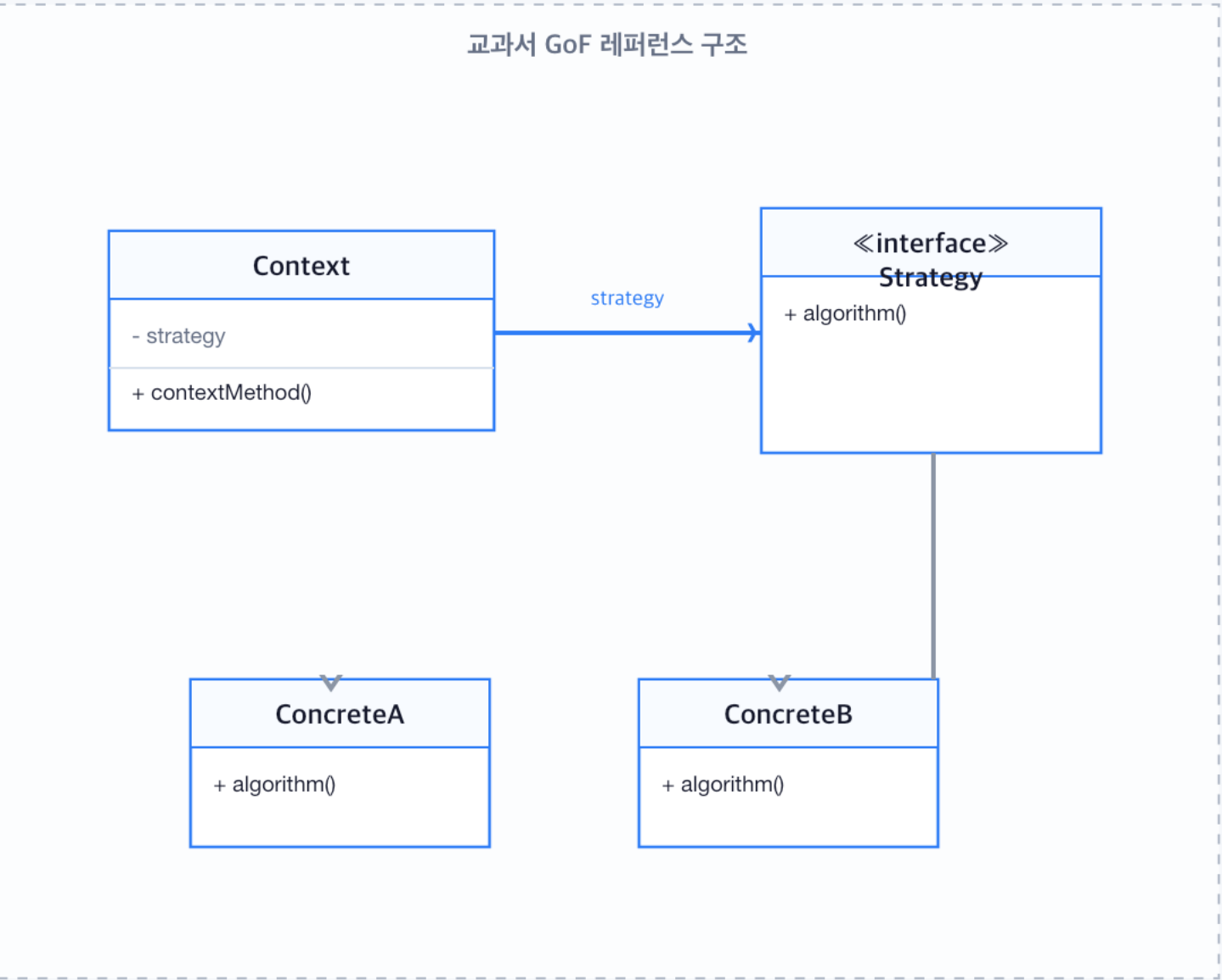
Full / Partial / No refund policies selected by FareRule and executed by RefundHandler

발표 핵심

paid는 추상 용어가 아니라 reservation.getPayments()에서 합산한 실제 결제 총액이다. 각 ConcreteStrategy가 이 paid를 다른 계산식으로 처리한다.

DP#2 Strategy — Reference vs Team

환불 금액 계산식을 정책 객체로 분리해 운임 규칙 변경에 대응한다.



DP#2 Strategy — Code Skeleton

예약 조회에서 취소/환불 미리보기로 policy 선택을 보여준다.

실제 코드 skeleton

```
interface RefundPolicy {
    BigDecimal calculateRefundAmount(BigDecimal paid);
    String getRefundType();
}

BigDecimal paid = reservation.getPayments().stream()
    .filter(p -> p.getAmount() != null)
    .map(Payment::getAmount)
    .reduce(BigDecimal.ZERO, BigDecimal::add);

RefundPolicy policy = resolvePolicy(fareRule);
BigDecimal refundAmount = policy.calculateRefundAmount(paid);

FullRefundPolicy    -> paid
PartialRefundPolicy -> paid.subtract(penalty)
NoRefundPolicy      -> BigDecimal.ZERO
```

논리 흐름



시연 멘트

화면에서 환불 미리보기 금액을 보여주고, 코드에서 paid 계산과 policy 호출을 설명한다.

Iteration 3 Recap — Observer

부수효과를 호출자에서 분리해 publisher/listener 구조로 이동했다.

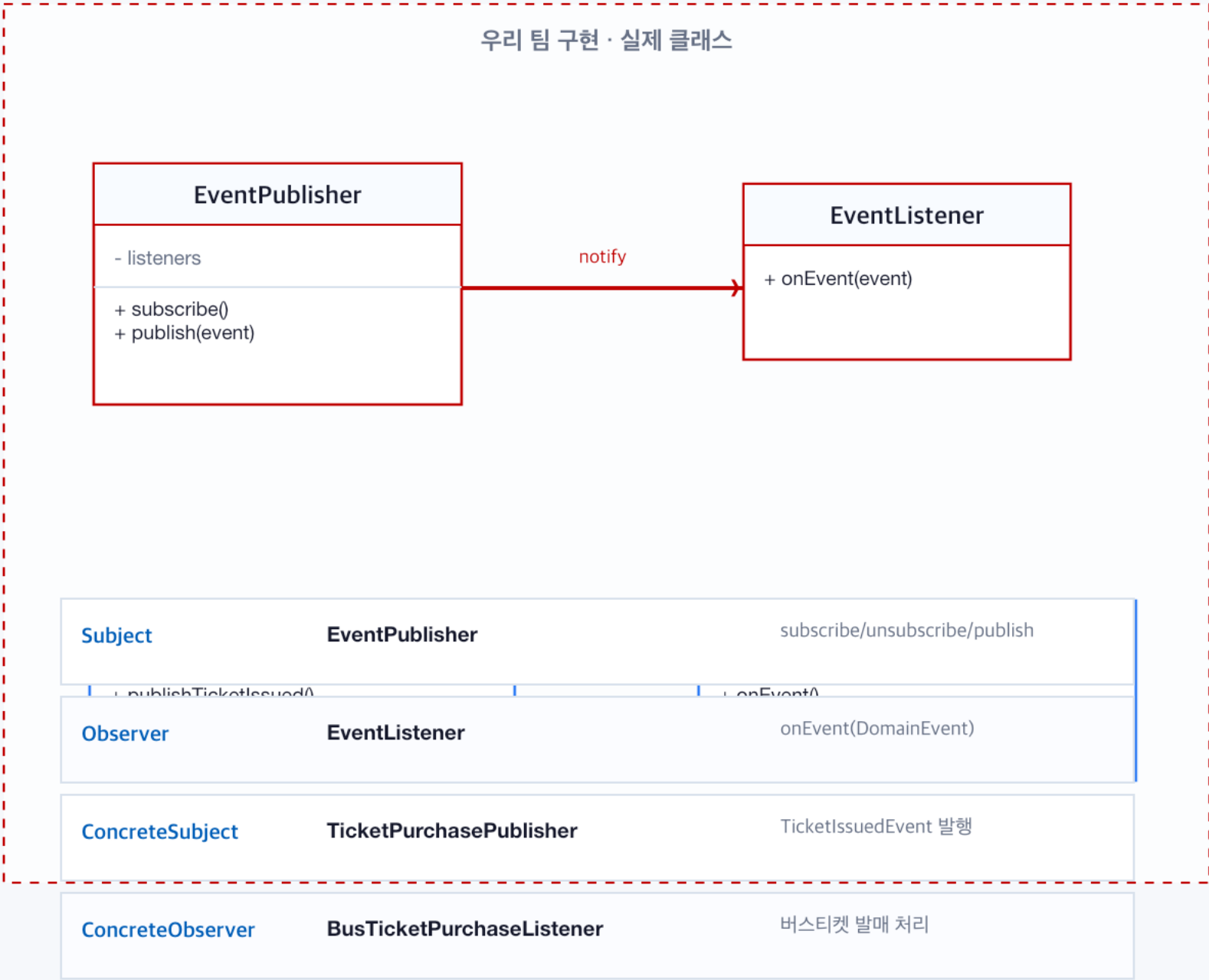
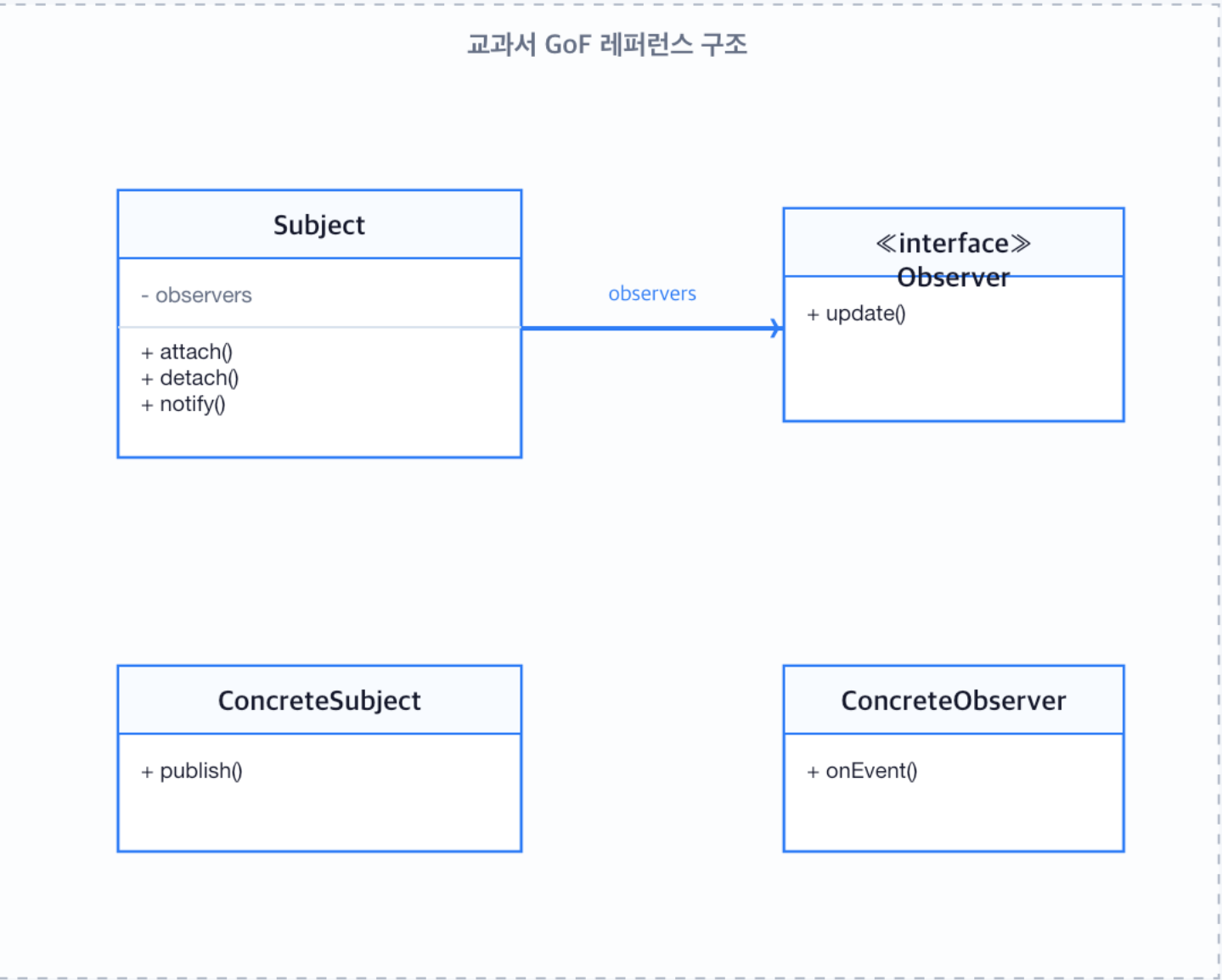
Subject	Event	Listener effect
SeatHoldMonitor	SeatHoldExpiredEvent	seat release + reservation cancel
PaymentProcessor	PaymentFailedEvent	auto-cancel on payment failure
FlightSchedule	FlightStatusChangedEvent	affected reservations update
TicketPurchasePublisher	TicketIssuedEvent	linked bus-ticket issue

발표 핵심

Observer는 iter3에서 도입됐지만 iter4 확인 화면의 버스티켓 연계 시연에서도 계속 active로 쓰인다.

DP#3 Observer — Reference vs Team

Subject는 이벤트만 발행하고 listener가 각자 관심 이벤트를 처리한다.



DP#3 Observer — Code Skeleton

발권 후 버스티켓 자동 발매 로그를 보여준다.

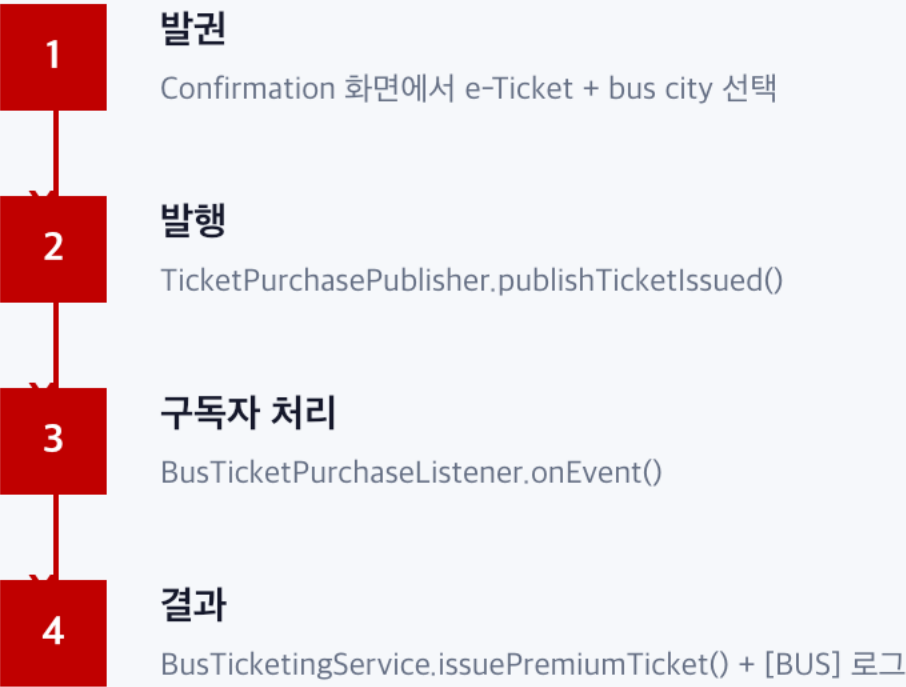
실제 코드 skeleton

```
class TicketPurchasePublisher extends EventPublisher {
  void publishTicketIssued(Reservation r, Ticket t, BusCity city) {
    publish(new TicketIssuedEvent(r, t, city));
  }
}

abstract class EventPublisher {
  private final List<EventListener> listeners = new ArrayList<>();
  void subscribe(EventListener l) { listeners.add(l); }
  protected void publish(DomainEvent event) {
    for (EventListener l : new ArrayList<>(listeners)) {
      l.onEvent(event);
    }
  }
}

class BusTicketPurchaseListener implements EventListener {
  public void onEvent(DomainEvent e) { issuePremiumTicket(...); }
}
```

논리 흐름

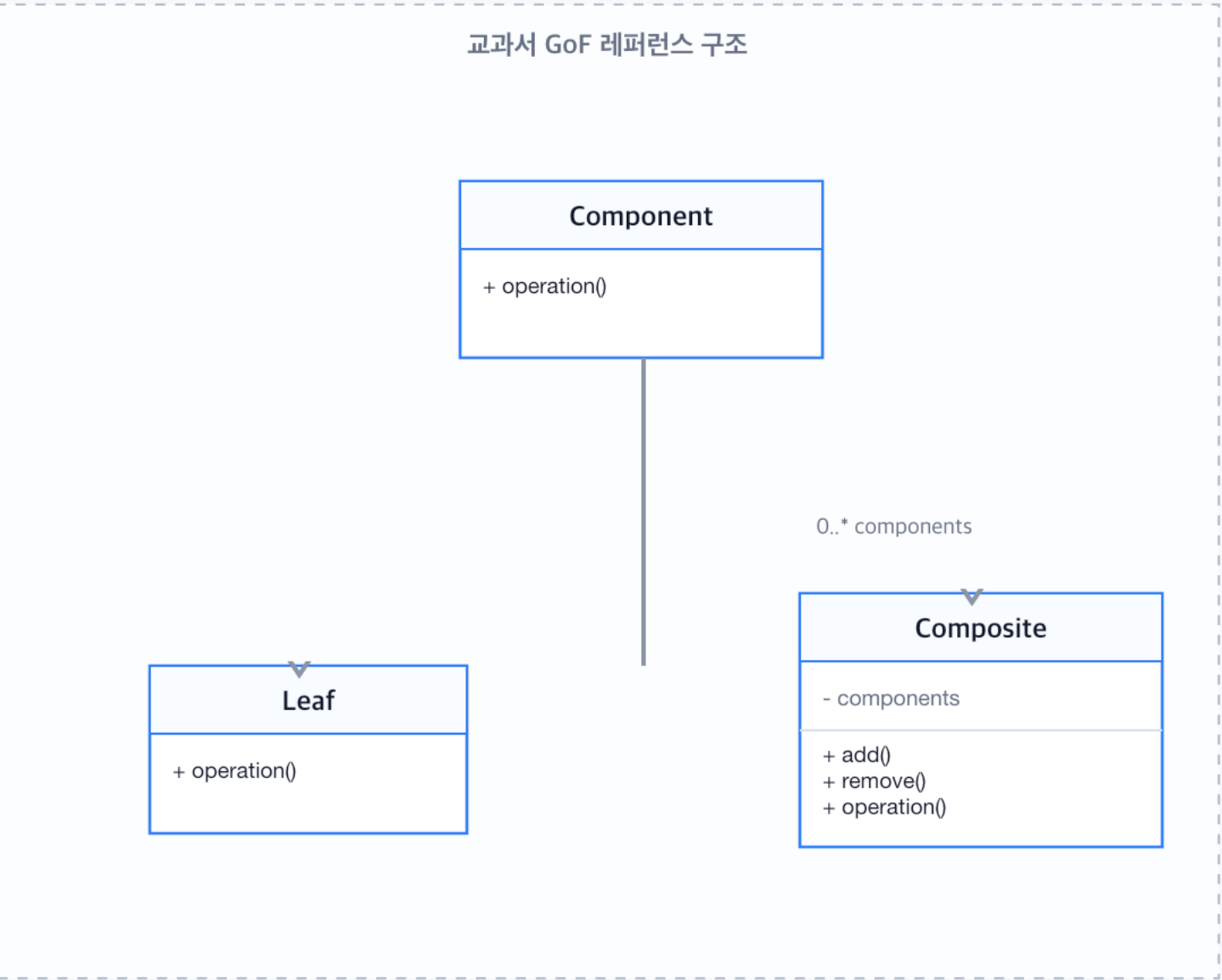


시연 멘트

터미널의 [BUS] 로그와 확인 화면의 버스티켓 상태를 같이 보여준다.

DP#4 Composite — Reference vs Team

도시와 공항을 AirportLocation 하나로 다뤄 city-code 검색을 확장한다.



DP#4 Composite — City-code Search

SEL/TYO/NYC/LON 검색을 여러 공항으로 확장한다.

실제 코드 skeleton

```
interface AirportLocation {
    String getCode();
    List<Airport> getAirports();
    boolean matches(String code);
    boolean isComposite();
}

class AirportCity implements AirportLocation {
    private final List<Airport> airports = new ArrayList<>();
    AirportCity add(Airport airport) { airports.add(airport); return this; }
    List<Airport> getAirports() { return unmodifiableList(airports); }
    boolean matches(String code) {
        return cityCode.equalsIgnoreCase(code)
            || airports.stream().anyMatch(a -> a.matches(code));
    }
}

FlightSearchService.searchByLocation(from, to, date)
```

논리 흐름

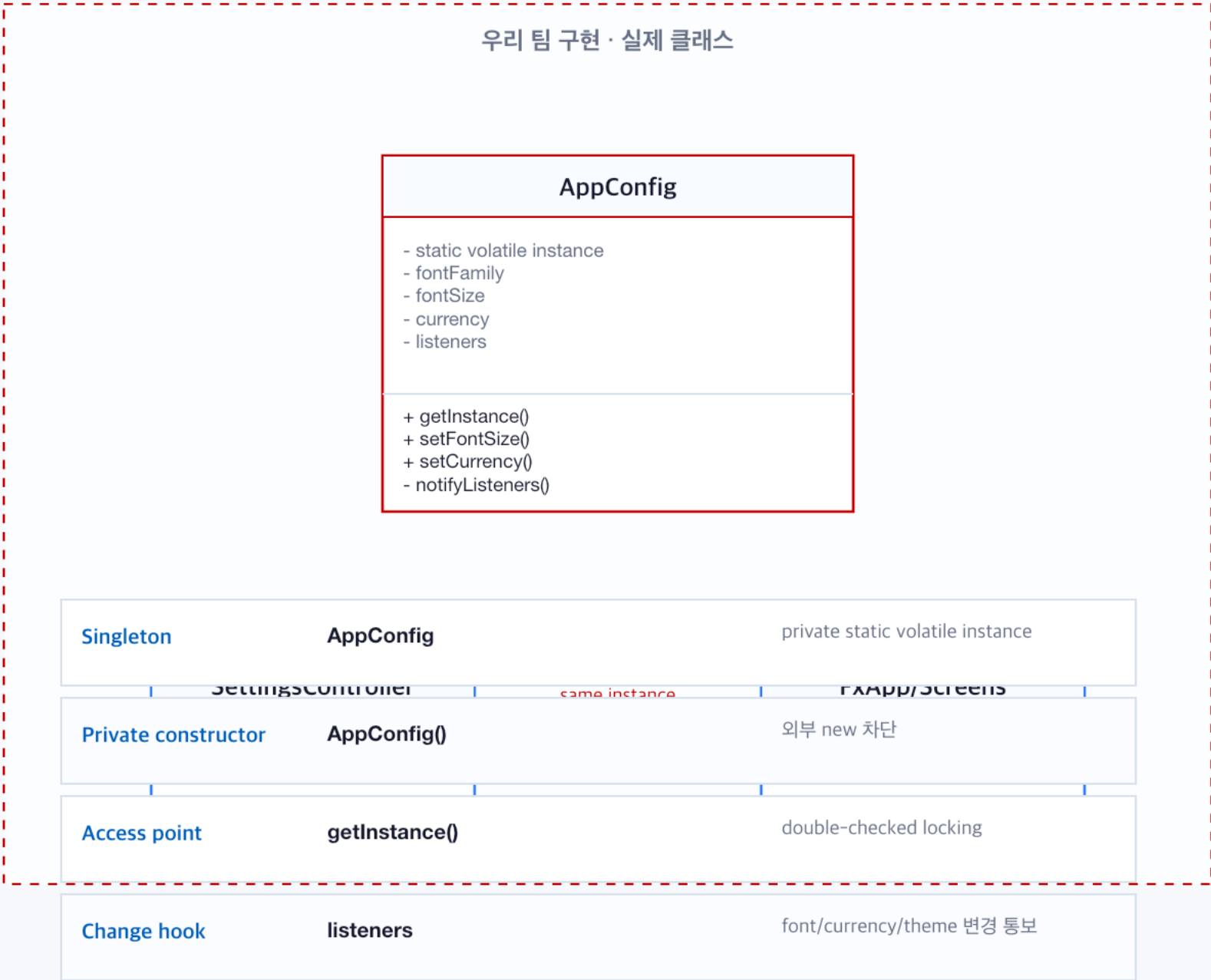


시연 멘트

TYO를 넣으면 NRT/HND 같은 소속 공항을 한 번에 검색한다고 설명한다.

DP#5 Singleton — Reference vs Team

전역 설정을 AppConfig 하나로 유지하고 listener로 화면을 재렌더한다.



DP#5 Singleton — AppConfig Flow

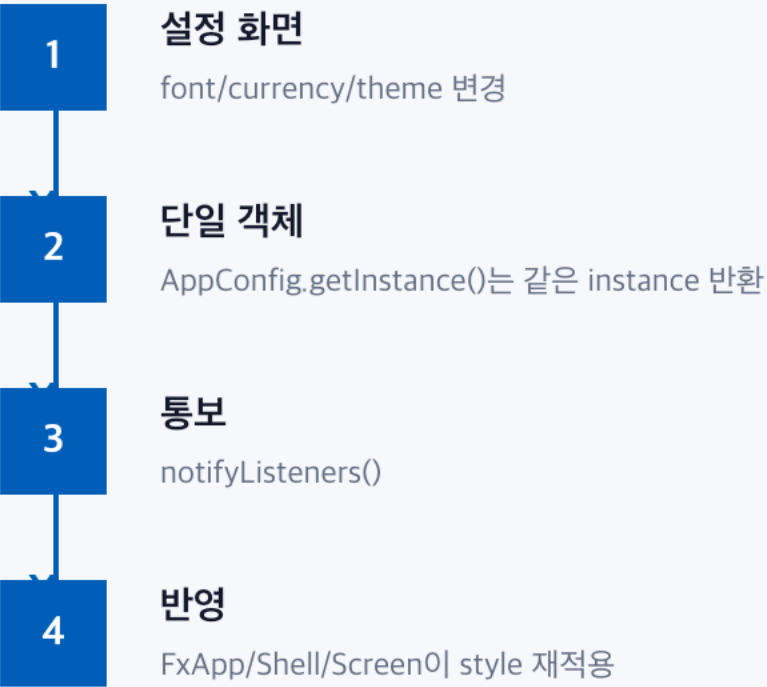
설정 변경이 같은 instance와 listener를 통해 모든 화면에 반영된다.

실제 코드 skeleton

```
public final class AppConfig {
    private static volatile AppConfig instance;
    private String fontFamily = "Pretendard";
    private int fontSize = 13;
    private Currency currency = Currency.getInstance("KRW");
    private final List<Consumer<AppConfig>> listeners = new ArrayList<>();

    private AppConfig() {}
    public static AppConfig getInstance() {
        AppConfig local = instance;
        if (local == null) synchronized (AppConfig.class) {
            local = instance;
            if (local == null) instance = local = new AppConfig();
        }
        return local;
    }
    public void setFontSize(int fontSize) { this.fontSize = fontSize; notifyListeners(); }
}
```

논리 흐름

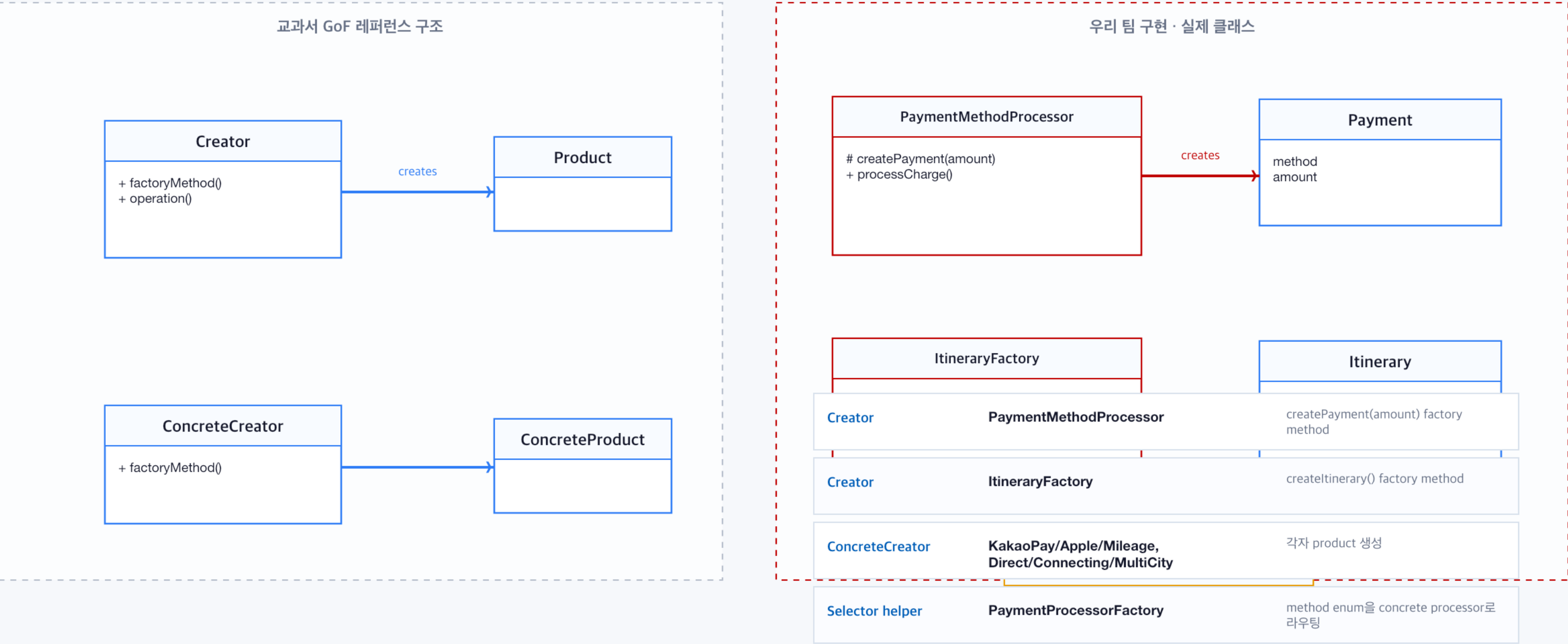


시연 멘트

Settings에서 글꼴/통화 변경 후 다른 화면에서도 유지되는지 보여준다.

DP#6 Factory Method — Reference vs Team

정확한 매핑: PaymentMethodProcessor와 ItineraryFactory가 Creator, static factory는 선택 helper다.



DP#6 Factory Method — Payment Processor

결제 수단별 processor가 자기 Payment를 생성한다.

실제 코드 skeleton

```
abstract class PaymentMethodProcessor extends EventPublisher {
    protected abstract Payment createPayment(long amount);
    public abstract PaymentMethod method();

    public Payment processCharge(long amount, String pnr) {
        Payment payment = createPayment(amount);
        boolean authorized = authorize(payment);
        if (authorized) payment.pay();
        else {
            payment.fail();
            publish(new PaymentFailedEvent(payment, pnr, declineReason()));
        }
        return payment;
    }
}

class KakaoPayPaymentProcessor extends PaymentMethodProcessor {
    protected Payment createPayment(long amount) { return basePayment(amount, KAKAO_PAY); }
}
```

논리 흐름



시연 멘트

Factory Method와 Observer가 결제 실패 흐름에서 연결되는 점까지 설명한다.

DP#6 Factory Method — ItineraryFactory

Direct/Connecting/MultiCity가 같은 build 흐름에서 다른 Itinerary를 생성한다.

실제 코드 skeleton

```
abstract class ItineraryFactory {
    public final Itinerary build(List<FlightSchedule> schedules) {
        validate(schedules);
        Itinerary itinerary = createItinerary();
        for (FlightSchedule fs : schedules) itinerary.addSegment(new Segment(fs));
        postCheck(itinerary);
        return itinerary;
    }
    protected abstract Itinerary createItinerary();
    protected abstract void validate(List<FlightSchedule> schedules);
}

class ConnectingItineraryFactory extends ItineraryFactory {
    protected Itinerary createItinerary() {
        Itinerary it = new Itinerary();
        it.setTripType("CONNECTING");
        return it;
    }
}
```

논리 흐름

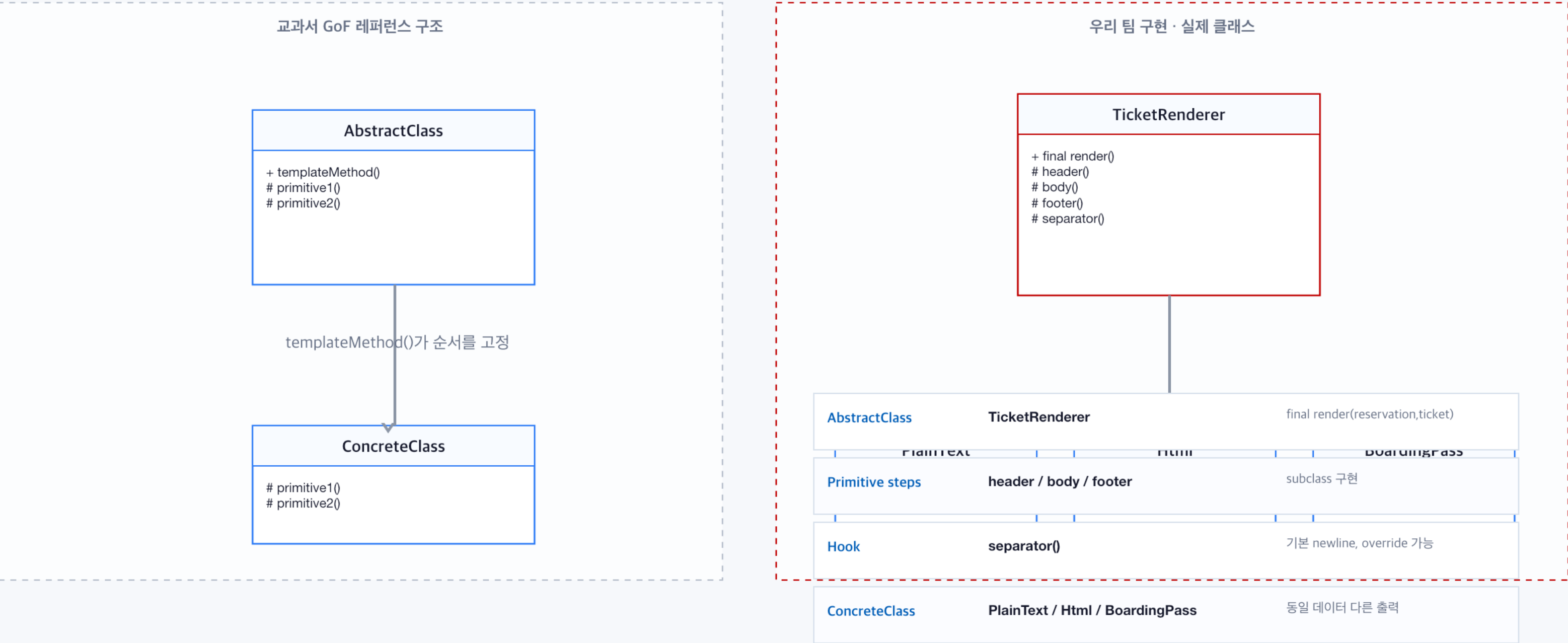


시연 멘트

환승 포함 검색을 켜고 Connecting itinerary가 factory로 만들어지는 장면을 설명한다.

DP#7 Template Method — Reference vs Team

e-Ticket 렌더링은 header-body-footer 순서가 고정되고 포맷만 달라진다.



DP#7 Template Method — TicketRenderer

확인 화면에서 같은 ticket data를 여러 포맷으로 렌더한다.

실제 코드 skeleton

```
public abstract class TicketRenderer {
    public final String render(Reservation reservation, Ticket ticket) {
        StringBuilder sb = new StringBuilder();
        sb.append(header(reservation, ticket));
        sb.append(separator());
        sb.append(body(reservation, ticket));
        sb.append(separator());
        sb.append/footer(reservation, ticket));
        return sb.toString();
    }
    protected abstract String header(Reservation r, Ticket t);
    protected abstract String body(Reservation r, Ticket t);
    protected abstract String footer(Reservation r, Ticket t);
    protected String separator() { return "\n"; }
}
```

논리 흐름

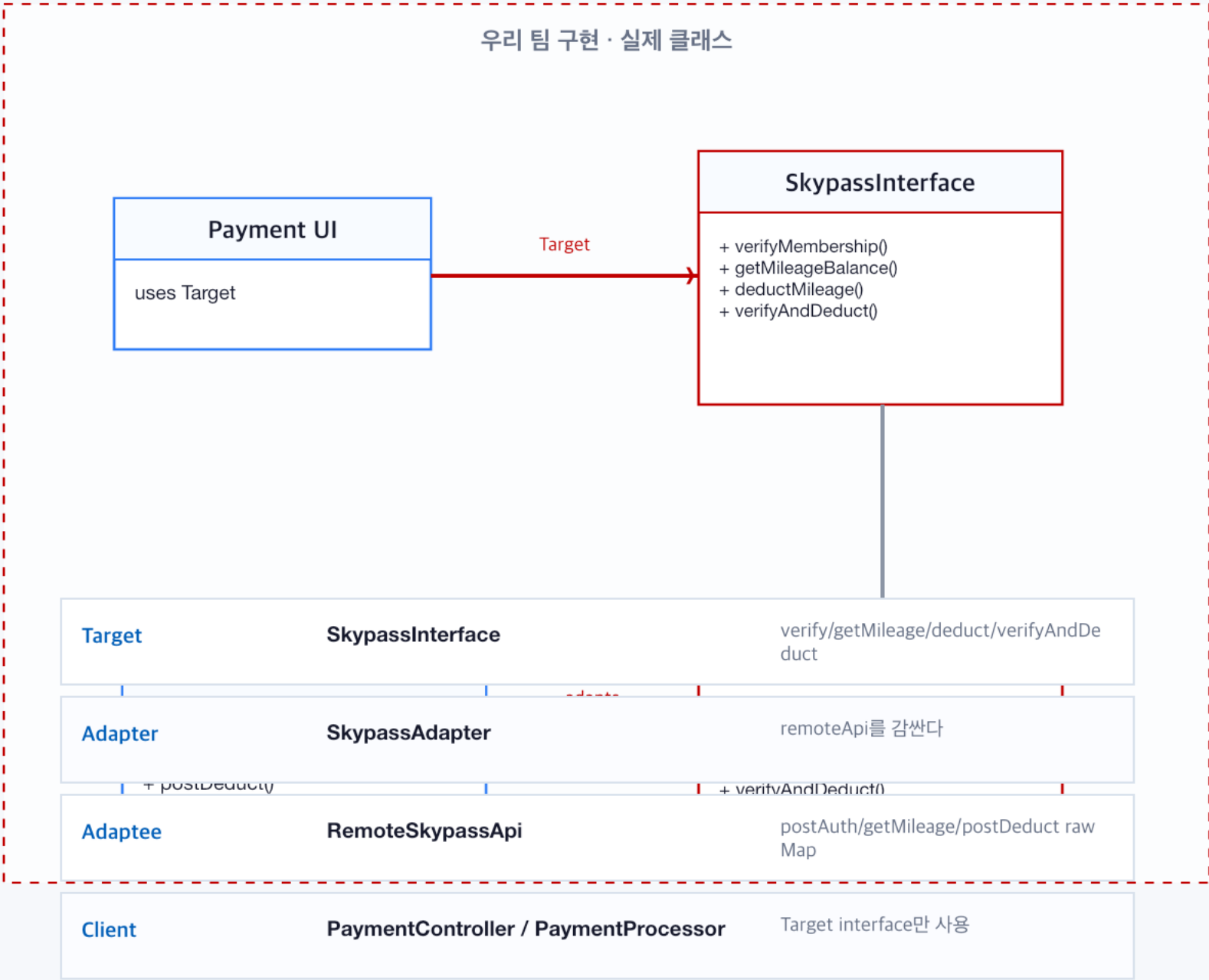
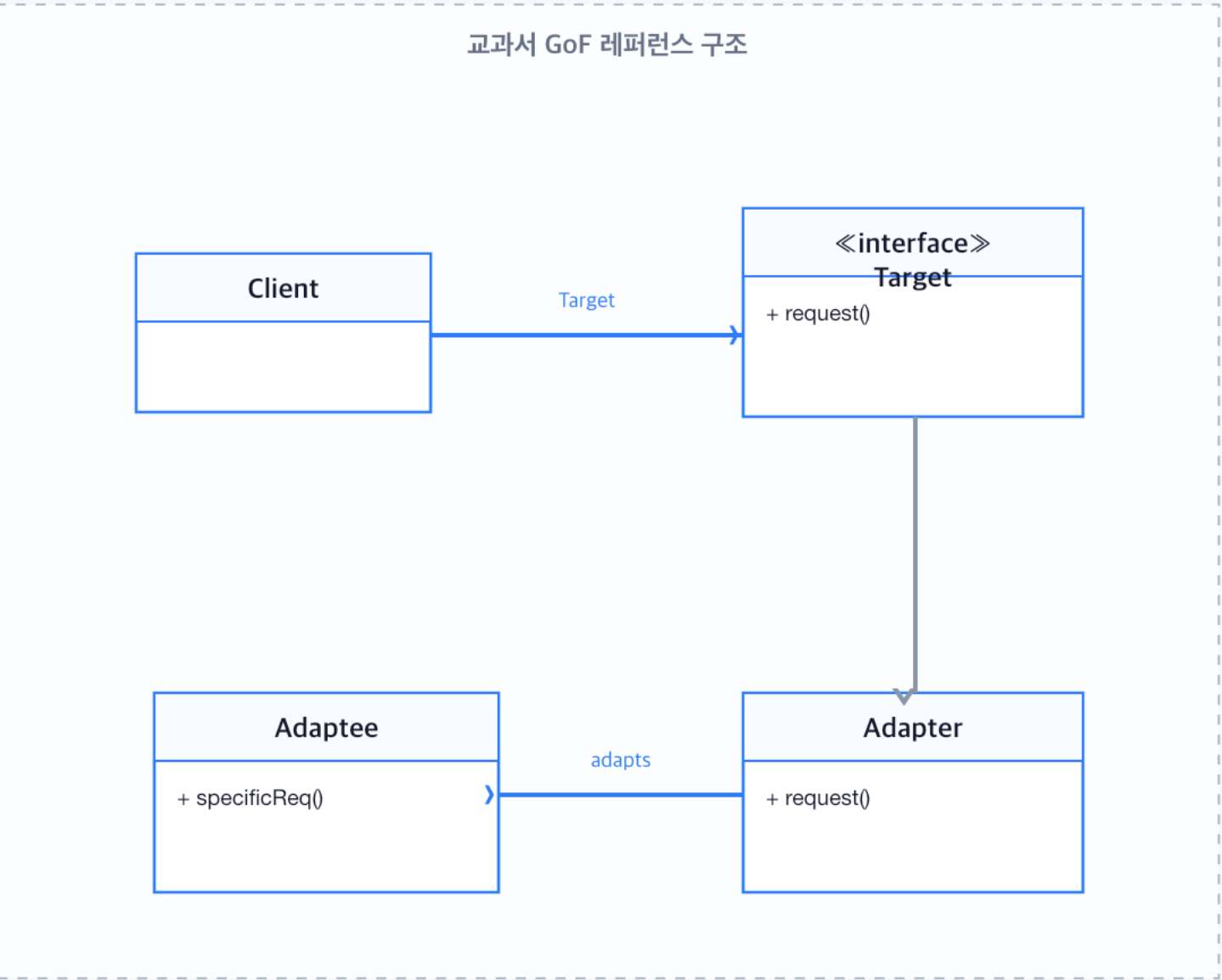


시연 멘트

확인 화면에서 e-Ticket 포맷 토글을 보여주며 순서 고정과 표현 변경을 설명한다.

DP#8 Adapter — Reference vs Team

외부 Skypass API의 raw Map 응답을 내부 SkypassInterface로 변환한다.



DP#8 Adapter — SkypassAdapter

결제 화면은 외부 API shape를 몰라도 마일리지 잔액과 차감을 쓴다.

실제 코드 skeleton

```
class SkypassAdapter implements SkypassInterface {
    private final RemoteSkypassApi remoteApi;

    public int getMileageBalance(String skypassNumber) {
        Map<String, Object> res = remoteApi.getMileage(skypassNumber);
        Object bal = res.get("balance");
        return bal instanceof Integer ? (Integer) bal : 0;
    }

    public Object verifyAndDeduct(String skypassNumber, int amount) {
        Map<String, Object> check = remoteApi.getMileage(skypassNumber);
        Object status = check.get("status");
        if (!(status instanceof Integer) || (Integer) status != 200) return null;
        return deductMileage(skypassNumber, amount) ? skypassNumber : null;
    }
}
```

논리 흐름

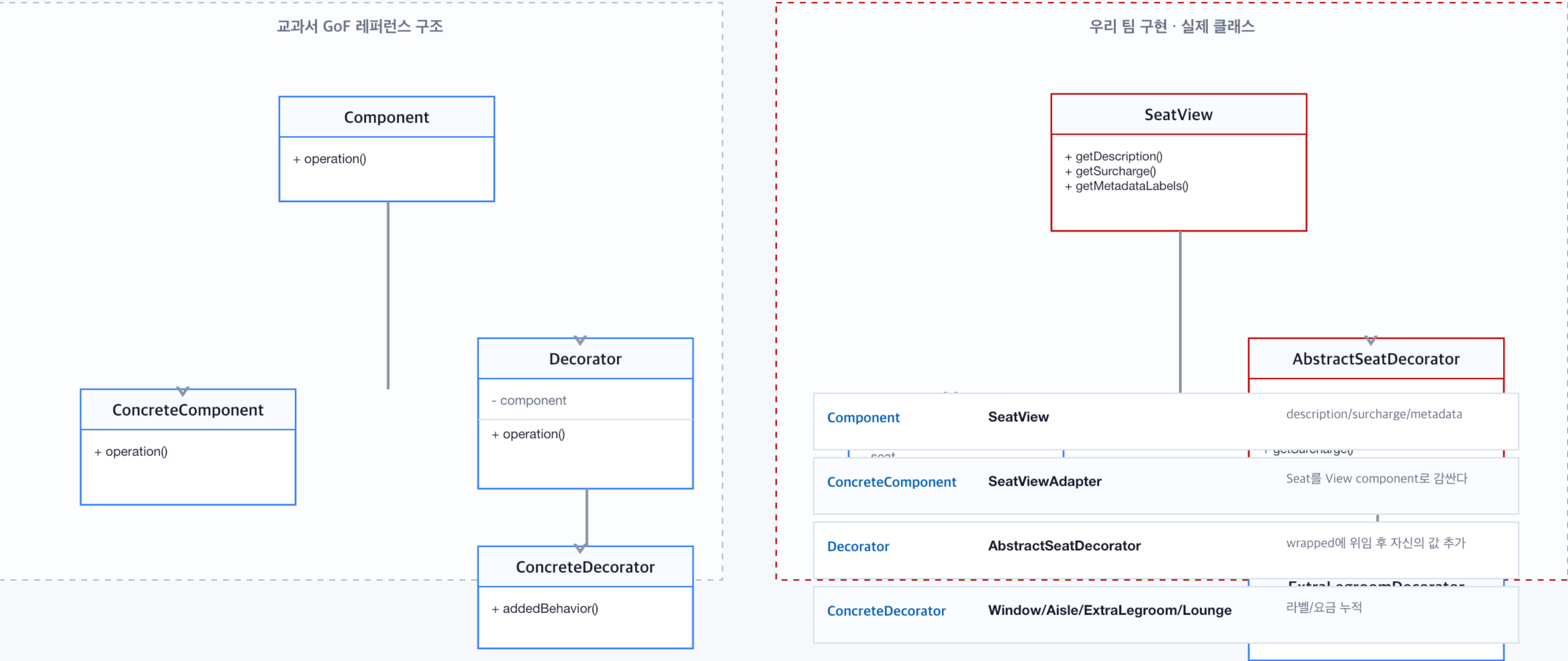


시연 멘트

결제 화면에서 마일리지 잔액과 차감이 보이는 부분을 Adapter로 설명한다.

DP#9 Decorator — Reference vs Team

좌석 옵션 조합을 class 폭발 없이 wrapper chain으로 누적한다.



DP#9 Decorator — SeatView Chain

좌석 화면에서 레그룸/라운지 선택이 결제 금액에 누적된다.

실제 코드 skeleton

```
public static SeatView decorate(Seat seat) {
    SeatView view = new SeatViewAdapter(seat);
    if (seat.isWindowSeat()) {
        view = new WindowSeatDecorator(view);
    } else if (seat.isAisleSeat()) {
        view = new AisleSeatDecorator(view);
    }
    if (seat.isExtraLegroom()) {
        view = new ExtraLegroomDecorator(view);
    }
    CabinClass cc = seat.getCabinClass();
    if (cc == BUSINESS || cc == FIRST) {
        view = new LoungeAccessDecorator(view);
    }
    return view;
}

getSurcharge() = wrapped.getSurcharge().add(extraSurcharge());
```

논리 흐름



시연 멘트

좌석 화면에서 추가 레그룸을 선택하면 50,000원이 합산되는 흐름을 보여준다.

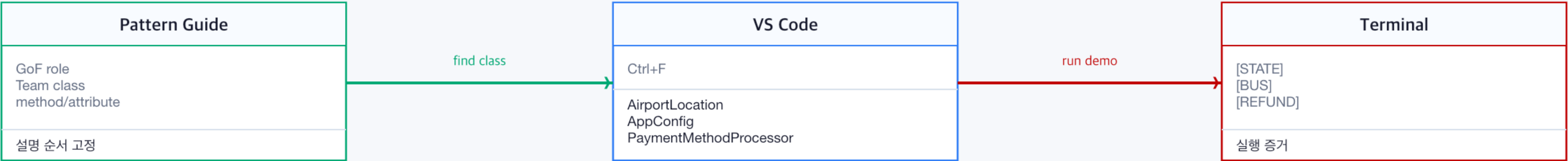
Section 9 Demonstration Scenarios

Google Doc의 SC-01~SC-06을 발표 순서로 정리한다.

ID	Scenario	Expected outcome	Pattern
SC-01	Guest booking end-to-end	State badge: PendingPayment → Confirmed → Ticketed	State
SC-02	Add legroom on seat screen	$320,000 + 32,000 + 50,000 = 402,000$	Decorator
SC-03	Change payment method	Matching processor selected	Factory Method
SC-04	Toggle e-ticket format	Same data rendered text/boarding pass	Template Method
SC-05	Change font in Settings	All screens re-render	Singleton
SC-06	Select shuttle city	Bus ticket auto-issued	Observer

Pattern Guide + VS Code 시연 방법

앱 안의 Pattern Guide를 목차처럼 사용하고, VS Code에서는 같은 클래스명을 Ctrl+F로 연다.



순서	발표 멘트
1	먼저 Pattern Guide에서 GoF 역할과 우리 클래스 대응을 보여준다.
2	VS Code에서 같은 클래스명을 열어 실제 method/attribute를 확인한다.
3	앱에서 버튼/선택을 누르고 터미널 로그나 화면 변화로 증명한다.

Section 10 Quality Assurance

문서에 있는 QA 항목과 실제 발표에서 말할 검증 근거.

Severity	Issue	Fix
HIGH	Double refund	Separated immediate cancel from manual review path
HIGH	Fare hardcoded for every reservation	Scaled fare and tax by itinerary segment count
HIGH	setPassengerInfo re-called on back navigation	Caught invalid transition and proceeded
HIGH	Approve double-refunded via processCancellation	Approve uses explicit transitions + single processRefund
MED	Connecting flight booked as multi-city	Added ConnectingItineraryFactory path
MED	Skypass number not incremented	Post-increment member sequence

검증 문장

AI code review 5회, Maven compile, 13 FXML runtime load, 수동 booking flow 검증을 근거로 말한다.

Section 11 Intentional Limits

못 만든 것을 숨기지 않고 범위 밖/후속 과제로 명확히 둔다.

Limit	Reason / wording
Admin console not built	authenticateAdmin, changeFlightStatus, createSchedule are backend stubs
Single adult, economy only	Booking flow is fixed to one adult, economy fare
Seat not shown on e-ticket	Seat reserved in inventory but ticket body link remains domain limitation
Real external systems	Skypass and payment are represented through Adapter/Gateway boundaries

발표 톤

한계는 실패가 아니라 설계 경계다. Adapter/Factory/Template이 후속 확장을 받을 수 있게 열어두었다고 설명한다.

Final Demo Runbook

시간이 부족해도 이 순서면 9개 DP가 모두 지나간다.

1. 검색

TYO/SEL 도시 코드
설명 DP: Composite

2. 좌석

A/F + legroom/lounge
설명 DP: Decorator

3. 결제

KakaoPay/Mileage
설명 DP: Factory + Adapter

4. 확정

e-Ticket + bus city
설명 DP: Template + Observer

5. 조회/환불

cancel/refund review
설명 DP: State + Strategy

6. 설정/가이드

font + Pattern Guide
설명 DP: Singleton + recap

터미널 로그

[STATE] Initiated -> PendingPayment
[STATE] PendingPayment -> Confirmed
[PLUS] premium bus ticket issued

Iteration 4 최종 변경사항 — 빨간색 표시

보고서/발표에서 빨간색으로 짚을 항목.

CHANGE	Swing Boundary를 JavaFX(FXML + CSS + Controller)로 교체
CHANGE	Composite, Singleton, Factory Method, Template Method, Adapter, Decorator 6개 GoF 패턴 추가
CHANGE	Iteration 1~4 누적 기준 9개 패턴이 모두 UI에서 시연 가능
CHANGE	Pattern Guide 화면으로 GoF 역할, 팀 클래스, method/attribute, skeleton, demo point를 확인
CHANGE	결제=Factory/Adapter, 좌석=Decorator, 확인=Template/Observer, 설정=Singleton으로 화면 연결

교수님 질문 대응 포인트

틀리기 쉬운 부분을 먼저 인정하고, 코드 기준으로 답할 문장을 준비한다.

질문 가능 지점	짧은 답변	근거 코드
Factory Method가 맞나?	Payment는 PaymentMethodProcessor.createPayment(), Itinerary는 ItineraryFactory.createItinerary()가 핵심 factory method입니다. PaymentProcessorFactory는 선택 helper입니다.	PaymentMethodProcessor.java / ItineraryFactory.java
State에 abstract class가 있나?	네. ReservationState interface 위에 AbstractReservationState가 기본 거부 동작을 제공해 중복을 줄입니다.	ReservationState.java / AbstractReservationState.java
UI 전환 때 Domain이 완전히 무변경인가?	절대 표현은 피하고, Boundary 중심 전환이며 Control/Domain의 핵심 패턴 구조를 보존했다고 말합니다.	app.fx package / BookingController glue
패턴이 앱에서 실제 호출되나?	검색, 좌석, 결제, 확정, 조회, 설정, Pattern Guide에서 각각 호출됩니다.	Search/Seat/Payment/Confirmation/Settings controllers

발표 태도

문서 문장을 외우기보다 실제 코드 클래스와 메서드로 설명한다. 애매한 용어는 'helper', 'Creator', 'ConcreteCreator'처럼 구분해서 말한다.

감사합니다

Iteration 4 Final · JavaFX + 9 Design Patterns

핵심 결론

기능이 늘어날수록 if문으로 버티는 대신,
변화 때마다 GoF 패턴으로 책임을 분리했다.

State · Strategy · Observer
Composite · Singleton · Factory Method
Template Method · Adapter · Decorator

Questions?